

## Module 8: Query Processing & Recovery

### ★ 8.1 How a Query Is Executed — Bonus • 10 min

#### Hook

You press run. Something reads your SQL, decides how to get the answer, and gets it.

That middle step — *deciding how* — is where the difference between 8 milliseconds and 40 seconds lives.

#### Why + What

SQL says **what** you want, never **how** to get it. So something has to invent the "how," and that something makes real choices: use an index or scan the table, join A to B or B to A, sort now or later.

**The query processor works in three stages: the parser checks your SQL, the optimiser picks the cheapest plan, and the executor runs it.**

#### How It Works (with the easiest example)

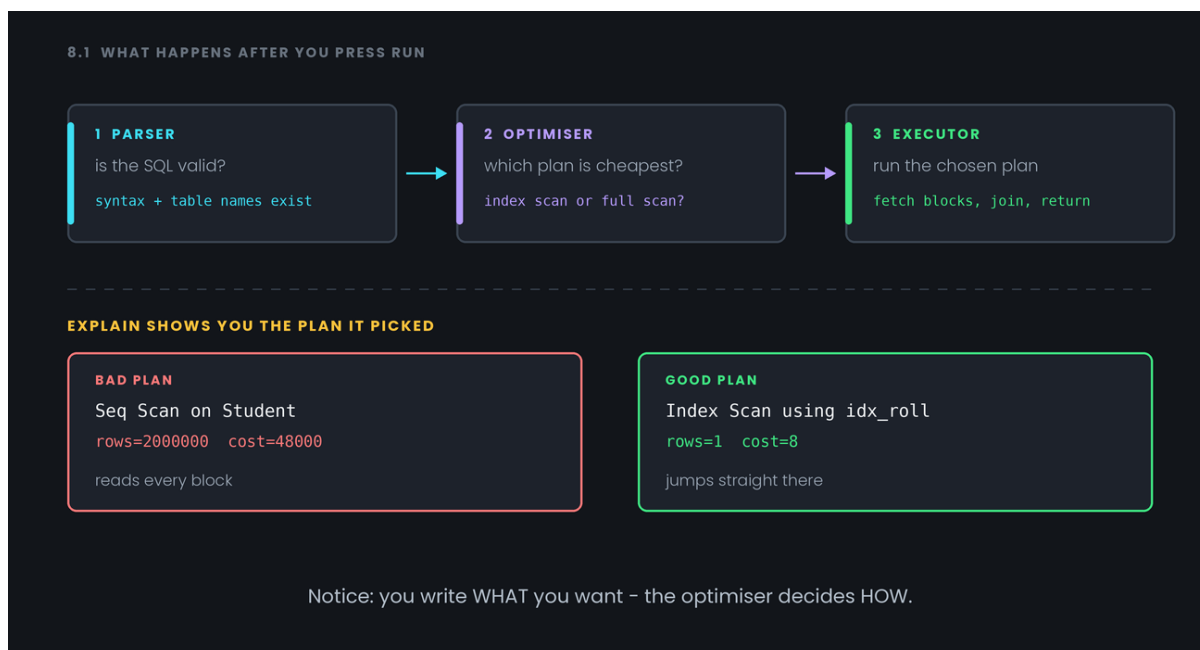
Take `SELECT name FROM Student WHERE roll_no = 101`.

1. **Parser.** Is the SQL valid? Does Student exist? Does it have roll\_no and name? Errors here are syntax and name errors — you've seen these already.
2. **Optimiser.** The interesting stage. It considers several plans: scan every block, or use the index on roll\_no. It **estimates the cost of each** using statistics it keeps about the table — how many rows, how many distinct values.
3. **Cost is measured in block reads**, which is exactly the idea from Module 7.1. Full scan of 2 million rows  $\approx$  25,000 block reads. Index lookup  $\approx$  4. It picks the index.
4. **Executor.** Runs the chosen plan and returns rows.
5. **See the plan yourself.** Put EXPLAIN in front of any query and the database prints what it decided. Seq Scan means full table scan. Index Scan means it used your index. That single word is usually the answer to "why is this slow."

**Pause here.** EXPLAIN shows Seq Scan on a query with a WHERE on an indexed column. Name two reasons it might have ignored the index.

Either the column is wrapped in a function (so the index doesn't match), or the query returns so much of the table that scanning is genuinely cheaper.

#### Visualize



*Notice: you write WHAT you want — the optimiser decides HOW.*

## Common Confusion

People think the optimiser is guaranteed to find the best plan. It isn't. It uses **estimates** from statistics, and stale statistics produce bad plans. That's why databases have a command to refresh them (ANALYZE in PostgreSQL) and why a query can suddenly get slow after a big data load without a single line of SQL changing.

Second: **cost numbers in EXPLAIN aren't seconds**. They're arbitrary units for comparing plans against each other. A cost of 48,000 doesn't mean 48 seconds — it means "much more work than the plan costing 8."

## Interview Angle

1. "What does a query optimiser do?" — Considers several execution plans and picks the cheapest by estimated cost, using table statistics. Say cost is measured in block reads.
2. "How would you debug a slow query?" — Run EXPLAIN, look for a sequential scan where an index should apply, check whether a function is disabling the index, and check whether statistics are stale.
3. "Are the numbers in EXPLAIN seconds?" — No, they're relative cost units for comparing plans.

**Recap:** A query goes parser → optimiser → executor, the optimiser estimates plan costs in block reads using table statistics, and EXPLAIN shows you which plan it chose.

---

## ★ 8.2 Join Algorithms — Bonus • 10 min

### Hook

In Module 5 you learned what a join *means*. You never learned how the database actually performs one.

There are three ways, and knowing which is running explains most join-related slowness.

### Why + What

"Match rows in A against rows in B" is a description, not a method. Compare every row against every row and a join of two 10,000-row tables is 100 million comparisons.

So databases have three strategies, and the optimiser picks one.

**Nested loop scans B for every row of A. Hash join builds a lookup table from A and probes it with B. Merge join sorts both sides and walks them together.**

### How It Works (with the easiest example)

Joining Student to Enrollment on roll\_no.

1. **Nested loop.** Take Aisha, scan all of Enrollment for roll 101. Take Ben, scan again. Simple, and the cost is roughly  $A \times B$  — brutal on big tables. But if Enrollment.roll\_no is indexed, each "scan" becomes an index lookup and it's suddenly excellent.
2. **Hash join.** Read Student once and build an in-memory lookup keyed on roll\_no. Then read Enrollment once, and for each row look up its match instantly. Cost is roughly  $A + B$  — dramatically better. **Only works for equality joins**, because hashing gives no ordering.
3. **Merge join.** Sort both sides by roll\_no, then walk the two sorted lists together like merging two decks of cards. Cost is **sort + A + B** — and if both sides are *already* sorted (say, both have indexes on the join column), the sort is free and this wins.
4. **Who chooses?** The optimiser, based on table sizes, available indexes and memory. **You don't pick this** — which is itself the point of the topic.
5. **What you can influence.** Add an index on the join column, and nested loop stops being catastrophic while merge join becomes cheap. Almost all join tuning is really index tuning.

### Visualize

## 8.2 THREE WAYS TO RUN A JOIN

### NESTED LOOP

for each row in A, scan B

cost  $A \times B$

best when: small tables, or B is indexed

### HASH JOIN

build a lookup table from A, probe with B

cost  $A + B$

best when: big tables, equality joins only

### MERGE JOIN

sort both, then walk them together

cost  $\text{sort} + A + B$

best when: already-sorted or indexed inputs

Notice: the optimiser picks one for you — it is not a setting you choose.

*Notice: the optimiser picks one for you — it is not a setting you choose.*

### Common Confusion

Candidates assume nested loop is always the bad one. It isn't. **With an index on the inner table, nested loop is often the fastest choice** — especially when the outer table is small. It's only terrible when both sides are big and unindexed.

Second: hash join needs memory to hold the lookup table. If it doesn't fit, the database spills to disk and performance drops sharply — one reason the same query can be fast on one server and slow on another.

### Interview Angle

1. "Name the join algorithms and when each is used." — Nested loop for small or indexed inner, hash join for big equality joins, merge join for already-sorted inputs.
2. "Why can't a hash join do a range join?" — Hashing destroys ordering, so  $<$  and BETWEEN can't be answered from a hash table.
3. "How do you make a join faster?" — Index the join columns, reduce rows before joining with a WHERE, and check EXPLAIN to see which algorithm was picked.

**Recap:** Nested loop rescans the inner side per row (great when indexed), hash join builds a lookup for equality joins, merge join walks two sorted inputs — and the optimiser chooses, not you.

---

## ★ 8.3 Recovery & the Write-Ahead Log — Bonus • 10 min

### Hook

Module 6 promised durability: once COMMIT returns, a crash can't undo your change.

But writing to a table means finding the right block on a slow disk. If the database did that before every commit, commits would crawl.

So how is it both fast *and* crash-proof?

### Why + What

By writing somewhere cheaper first.

**A write-ahead log (WAL) records what a transaction is about to change, and that log entry is written to disk before the actual data is. If a crash happens, the log is used to reconstruct the correct state.**

The log is fast to write because it's **sequential** — appended to the end of a file — while updating table blocks means jumping around the disk.

### How It Works (with the easiest example)

Aisha's fee payment again: 12,000 → 0.

1. **Write the log entry first.** "Transaction 7 changed Student 101 fees from 12000 to 0."  
Small, sequential, fast.
2. **Return the COMMIT.** Once the *log* is safely on disk, the transaction is durable. The user is told it succeeded.
3. **Update the real table block later.** The database does this at its leisure, batching writes efficiently. This is why commits feel instant.
4. **Now crash — and restart.** The database reads the log and does two passes:
  - **REDO** — the transaction committed but its data block was never written. Replay it. **Durability preserved.**
  - **UNDO** — the transaction never committed, but its data block *was* already written. Roll it back. **Atomicity preserved.**
5. **See what this means.** ACID's A and D from Module 6 aren't abstract promises. They're implemented by this log.

**Pause here.** Why is writing the log faster than writing the data block, when both go to the same disk?

The log is appended sequentially to the end of one file, so the disk writes in a straight line. Table blocks are scattered, so writing them means seeking around.

### Visualize

### 8.3 RECOVERY & THE WRITE-AHEAD LOG

#### THE RULE • LOG FIRST, DATA LATER

##### 1 WRITE THE LOG

fees: 12000 -> 0  
tiny, sequential, fast

##### 2 UPDATE THE DATA

the real table block  
slower, can wait

crash between 1 and 2?  
the log still has the truth.

#### ON RESTART, THE DATABASE READS THE LOG TWICE

##### REDO

transaction reached COMMIT  
but data was not written yet  
-> replay it. durability kept.

##### UNDO

transaction never committed  
but data was already written  
-> roll it back. atomicity kept.

Notice: the log is what makes the D and the A in ACID actually work.

Notice: the log is what makes the D and the A in ACID actually work.

### Common Confusion

"Write-ahead" confuses people. It means the **log is written ahead of the data**, not that anything is written ahead of time. Log first, data after. That ordering is the entire guarantee.

Second: people assume COMMIT waits for the data to be saved. It doesn't — it waits for the **log entry** to be saved. The data catches up afterwards. That's the whole performance trick.

### Interview Angle

1. "What is a write-ahead log?" — A record of intended changes written to disk before the data itself, used to recover after a crash. Say it's sequential and therefore fast.
2. "What are redo and undo?" — Redo replays committed transactions whose data never landed; undo rolls back uncommitted ones whose data did.
3. "How does a database make COMMIT fast and still durable?" — It only waits for the log write, not the data write.

**Recap:** A write-ahead log records changes before the data is updated, making commits fast and durable — and on restart, redo replays committed work while undo rolls back uncommitted work.

## ★ 8.4 Checkpoints & Backups — Bonus • 10 min

### Hook

Your database has been running for two years. Its log is enormous.

A crash happens. Does it really replay two years of history to start up?

## Why + What

It would, without checkpoints. And "restarts in six hours" is not an acceptable database.

**A checkpoint is a marker saying "everything before this point has been written to the data files." Recovery only needs to replay the log from the last checkpoint.**

Checkpoints handle crashes. They don't handle a **dead disk** — for that you need a copy elsewhere.

**A backup is a copy of the database stored separately, so you can rebuild after losing the storage itself.**

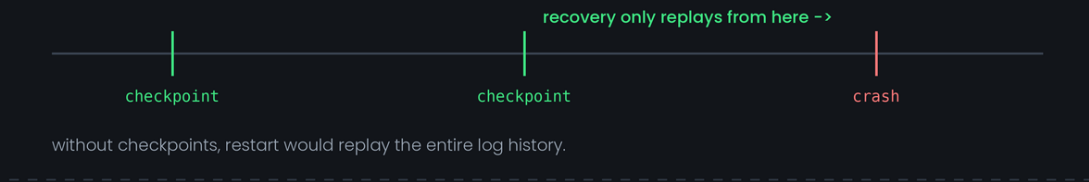
## How It Works (with the easiest example)

1. **The database checkpoints periodically.** It flushes pending changes to the data files and writes a marker in the log.
2. **A crash happens later.** Recovery starts at the **last checkpoint**, not the beginning of time. Everything before it is already safely in the data files. Restart takes seconds instead of hours.
3. **Now the disk itself dies.** Checkpoints are useless — they lived on the dead disk. You need the backup.
4. **Three backup styles:**
  - **Full** — everything, every time. Slow to take, simple and fast to restore.
  - **Incremental** — only what changed since the **last backup of any kind**. Fast to take; restoring means replaying the full plus every increment in order.
  - **Differential** — everything changed since the last **full** backup. Middle ground: bigger than incremental, but restore needs only two pieces.
5. **Pick by which pain you prefer.** Nightly incrementals are cheap to take and painful to restore. Full backups are the reverse. Most real setups combine them: weekly full, daily incremental.

## Visualize

## 8.4 CHECKPOINTS & BACKUPS

### A CHECKPOINT MARKS HOW FAR THE LOG HAS BEEN APPLIED



### THREE KINDS OF BACKUP

|                                                                        |                                                                                                |                                                                                        |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>FULL</b><br>everything, every time<br>slow to take, fast to restore | <b>INCREMENTAL</b><br>only what changed since the LAST backup<br>fast to take, slow to restore | <b>DIFFERENTIAL</b><br>everything changed since the last FULL<br>middle ground on both |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|

Notice: a checkpoint shortens recovery - a backup survives losing the disk.

*Notice: a checkpoint shortens recovery — a backup survives losing the disk.*

### Common Confusion

**A checkpoint is not a backup.** A checkpoint lives inside the same database on the same disk and only shortens crash recovery. A backup is a separate copy that survives the disk dying. Losing this distinction is a reliable way to fail a question.

Second: **an untested backup isn't a backup.** The failure everyone actually hits isn't a missing backup — it's a backup nobody ever tried restoring, discovered to be corrupt on the worst possible day. Saying this out loud in an interview lands well, because it's what operations people care about.

### Interview Angle

1. *"What is a checkpoint and why does it exist?"* — A marker showing the log has been applied up to that point, so recovery replays only from there rather than from the beginning.
2. *"Difference between full, incremental and differential backups?"* — Full copies everything; incremental copies changes since the last backup; differential copies changes since the last full. Trade-off is take-time versus restore-time.
3. *"Is a checkpoint a backup?"* — No. A checkpoint shortens crash recovery on the same disk; a backup survives losing that disk.

**Recap:** Checkpoints mark how far the log has been applied so crash recovery is short, while backups are separate copies that survive losing the storage — full, incremental and differential trading take-time against restore-time.

## Module 9: Beyond Relational

*This module is context, not core interview material. It exists so that when someone says "why not just use MongoDB," you have a real answer instead of a shrug.*

*Same story throughout: student **Aisha** (roll 101) takes **DBMS 101**, taught by **Prof. Rao**, and gets grade **A**.*

---

### ★ 9.1 SQL vs NoSQL — Bonus • 9 min

#### Hook

Seven modules of tables, keys and normalization. Then someone in the room says:

*"Why bother? We just use MongoDB. No schema, no joins, way faster."*

Is he right?

#### Why + What

Partly — and the honest answer is more interesting than either side of the usual argument.

Relational databases make you decide the shape of your data **before** you store any, and in return they enforce it forever. That's a genuine cost when the shape is still changing, or when one machine can't hold your data.

**NoSQL is a family of databases that drop some relational guarantees — fixed schema, joins, sometimes strict consistency — in exchange for flexibility and easier scaling across many machines.**

Note the word **drop**. Everything NoSQL gains, it gains by giving something up.

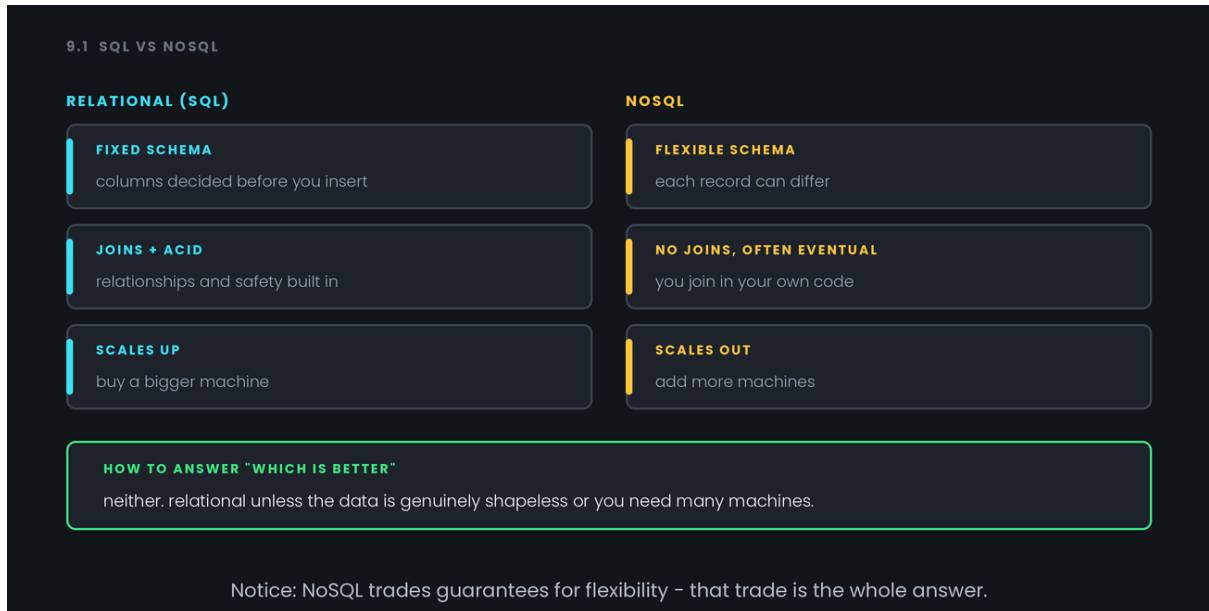
#### How It Works (with the easiest example)

Store Aisha's record both ways.

1. **Relational.** Student(roll\_no, name, email) and Enrollment(roll\_no, course, grade). The columns are fixed. Adding a phone number means altering the table. Getting her grades means a join.
2. **Document (NoSQL).** One JSON document holding Aisha *and* her enrollments nested inside it. Reading everything about her is **one** lookup, no join. Adding a phone number means... just adding it, to her document only.
3. **Now the catch.** Ben's document might not have a phone number. Or might spell the field differently. Nothing stops it — **the database enforces nothing**. Every guarantee from Module 3 is now your application's job.
4. **The scaling difference.** Relational databases traditionally **scale up** — buy a bigger machine. NoSQL systems are built to **scale out** — add more machines. Joins and cross-machine transactions are exactly what makes scaling out hard, so dropping them is what makes it easy.

5. **The honest answer.** Use relational unless your data is genuinely shapeless, or you genuinely need many machines. Most student projects and most company applications are neither.

## Visualize



*Notice: NoSQL trades guarantees for flexibility — that trade is the whole answer.*

## Common Confusion

**"NoSQL is faster" is not a real claim.** Faster at what? A document store is faster at fetching one whole nested record. A relational database is faster at "average grade per course across 2 million enrollments." They're fast at different shapes of question.

Second: **"NoSQL" doesn't mean "no SQL."** It's usually read as "not only SQL," and several NoSQL systems now offer SQL-like query languages. The name is historical, not descriptive.

Third: **schema-less doesn't mean no schema.** The schema still exists — it just moved into your application code, where nothing enforces it. Many teams discover this the hard way, three years in.

## Interview Angle

1. *"When would you choose NoSQL over a relational database?"* — Genuinely variable record shapes, very high write volume, or data too large for one machine. Say relational is the default and NoSQL needs a reason.
2. *"What do you give up with NoSQL?"* — Enforced schema, joins, and often strict consistency. Those guarantees move into your application.
3. *"Is NoSQL faster than SQL?"* — Depends on the question shape. Faster for whole-record fetches, usually slower for aggregate analysis across relationships.

**Recap:** NoSQL drops schema enforcement, joins and sometimes strict consistency to gain flexibility and horizontal scaling — so relational stays the default unless one of those trades genuinely pays.

---

## ★ 9.2 The Four NoSQL Families — Bonus • 9 min

### Hook

"NoSQL" is not one thing. Redis and Neo4j are both NoSQL, and they have almost nothing in common.

Four families, four shapes of data.

### Why + What

Each family exists because a particular data shape was awkward in tables. Knowing which is which means you can name the right tool instead of saying "some NoSQL thing."

**The four families are key-value, document, column-family and graph — each built for a different shape of data.**

### How It Works (with the easiest example)

1. **Key-value.** One key, one blob of data. That's all. session:abc123 → the logged-in user's session. Blindingly fast, no querying inside the value. *Redis*. Good for caches, sessions, counters.
2. **Document.** A record is a JSON document, and it can nest. Aisha plus all her enrollments inside one document. You *can* query inside it, unlike key-value. *MongoDB*. Good for catalogues, user profiles, content.
3. **Column-family.** Rows that can have thousands of sparse columns, stored by column rather than by row. Built for enormous write volume. *Cassandra*. Good for time series, event logs, sensor data.
4. **Graph.** Nodes and the edges between them are both first-class. "Find friends of friends who took DBMS 101" is a short query here and a nightmare of self-joins in SQL. *Neo4j*. Good for social networks, fraud detection, routing.
5. **Match family to shape.** Flat lookups → key-value. Nested records → document. Massive write streams → column-family. Heavily connected data → graph.

### Visualize

## 9.2 THE FOUR NOSQL FAMILIES

### KEY-VALUE

one key, one blob

good for: sessions, caches

Redis

### DOCUMENT

JSON records, nested

good for: catalogues, profiles

MongoDB

### COLUMN-FAMILY

rows with huge sparse columns

good for: time series, logs

Cassandra

### GRAPH

nodes and edges are first class

good for: social, fraud, routes

Neo4j

Notice: pick the family by the shape of your data, not by popularity.

*Notice: pick the family by the shape of your data, not by popularity.*

### Common Confusion

People use "NoSQL" and "MongoDB" interchangeably. MongoDB is one product in one family. Naming the four families and one example of each is a small thing that separates a real answer from a vague one.

Second: **graph databases aren't for "graphs" in the chart sense.** They're for networks of connections — people, accounts, roads. The name confuses a surprising number of candidates.

### Interview Angle

1. *"Name the types of NoSQL databases with an example of each."* — Key-value (Redis), document (MongoDB), column-family (Cassandra), graph (Neo4j). Add one use case each.
2. *"When is a graph database clearly the right choice?"* — When queries follow relationships several hops deep, like friends-of-friends, where SQL would need repeated self-joins.
3. *"What is a key-value store good for?"* — Fast lookups by a single key — sessions, caches, counters — where you never need to query inside the stored value.

**Recap:** Key-value stores one blob per key, document stores nested records, column-family handles huge sparse write-heavy data, and graph makes connections first-class.

---

## ★ 9.3 The CAP Theorem — Bonus • 9 min

### Hook

Your database now runs on three machines. The network cable between two of them fails.

A request arrives. Do you answer with possibly-stale data, or refuse to answer at all?

There is no third option. That's the CAP theorem.

### Why + What

The moment data lives on more than one machine, the machines can disagree. CAP names exactly what you must give up when they do.

**CAP says a distributed system cannot have all three of: Consistency (every read gets the latest write), Availability (every request gets an answer), and Partition tolerance (it keeps working when the network splits).**

### How It Works (with the easiest example)

Aisha's fees are stored on machines M1 and M2. The link between them breaks.

1. **A payment arrives at M1.** M1 sets her fees to 0. It cannot tell M2 — the link is down.
2. **Someone reads her fees from M2.** M2 still says 12,000. It has two options and no others.
3. **Option A — choose Consistency.** M2 refuses to answer, because it can't be sure it's current. Correct, but **unavailable**. This is **CP**.
4. **Option B — choose Availability.** M2 answers 12,000, which is stale. Available, but **wrong**. This is **AP**.
5. **Why P isn't optional.** Networks fail — cables, switches, cloud zones. A distributed system that can't tolerate a partition is a system that breaks. So **P is forced**, and the real choice is only ever **C or A**.

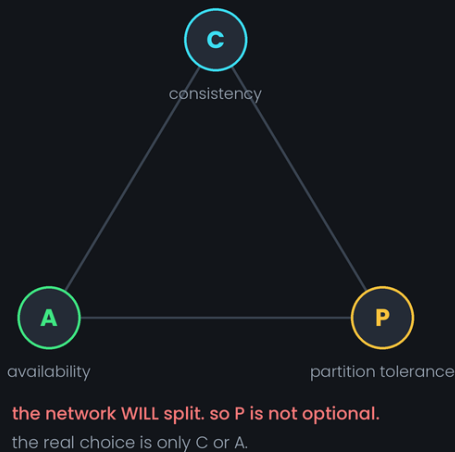
Pick by the cost of being wrong. Bank balances, seat bookings, inventory → **CP**: refuse rather than answer wrongly. Like counts, feeds, product listings → **AP**: a slightly stale number is fine, an error page is not.

**Pause here.** Two people book the last seat on a flight during a network partition. Which side would you choose, and what happens if you choose the other?

CP — refuse the booking rather than sell one seat twice. Choose AP and both bookings succeed, and someone gets turned away at the gate.

### Visualize

### 9.3 THE CAP THEOREM



#### CP · CHOOSE CONSISTENCY

refuse the request rather than answer wrongly

banking, inventory, bookings

#### AP · CHOOSE AVAILABILITY

answer anyway, possibly with stale data

feeds, likes, product catalogues

Notice: CAP is not pick-two — P is forced, so you are only ever picking C or A.

Notice: CAP is not pick-two — P is forced, so you are only ever picking C or A.

### Common Confusion

**"Pick any two" is the phrase everyone repeats, and it's misleading.** You don't get to drop P — the network will partition whether you like it or not. So CA isn't a real option for a distributed system; it only describes a single machine.

Second: **CAP only applies while a partition is happening.** When the network is healthy, a system can be both consistent and available. CAP describes the failure case, not everyday operation.

Third: **"eventual consistency" isn't broken.** It means the machines will agree once the partition heals — often within milliseconds. For a like counter that's completely fine. For a bank balance it isn't. The judgement is the answer, not the label.

### Interview Angle

1. *"Explain the CAP theorem."* — Consistency, availability, partition tolerance; you can't have all three during a partition. Then the strong follow-up: P isn't optional, so the real choice is C or A.
2. *"Give a system that chooses each."* — CP for banking, inventory, bookings. AP for social feeds, catalogues, analytics.
3. *"What is eventual consistency?"* — Copies converge once the partition heals. Acceptable when a slightly stale read is harmless.

**Recap:** During a network partition a distributed system must choose consistency or availability, never both — and since partitions are unavoidable, CAP is really a choice between refusing to answer and answering with stale data.

## ★ 9.4 Sharding & Replication — Bonus • 9 min

### Hook

Your college database outgrows one machine.

You can split the data across several machines, or copy it to several machines. Those sound similar. They solve completely different problems.

### Why + What

One machine has a size limit and a speed limit, and it's also a single thing that can die. Two different fixes:

**Sharding splits the rows across machines, so each holds a different slice. Replication copies the same rows to several machines, so each holds the same data.**

Split versus copy. Everything follows from that.

### How It Works (with the easiest example)

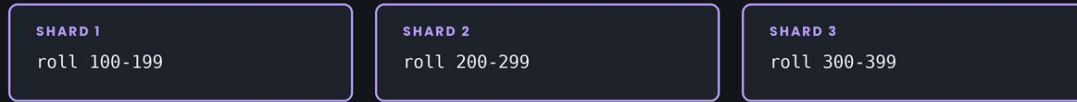
2 million students, one machine straining.

1. **Shard by roll number.** Rolls 100–199 on machine 1, 200–299 on machine 2, 300–399 on machine 3. Each machine now holds a third of the data and a third of the load. **Capacity solved.**
2. **The cost of sharding.** A query for one student is easy — go to the right shard. But "average grade across all students" must visit **every** shard and combine results. And a transaction spanning two shards is genuinely hard. Choosing a bad **shard key** — one that sends most traffic to one machine — recreates the original problem.
3. **Now replicate instead.** One **primary** takes all writes. Two **replicas** hold copies and serve reads. Read-heavy workloads get much faster, because reads spread across three machines.
4. **The cost of replication.** Replicas lag behind the primary by a little. A read right after a write may return the old value — which is the **eventual consistency** from 9.3, appearing in a real system.
5. **Most real systems do both.** Shard for capacity, replicate each shard for read speed and safety. And replication doubles as a spare copy: if the primary dies, a replica takes over.

### Visualize

#### 9.4 SHARDING & REPLICATION

##### SHARDING · SPLIT THE ROWS ACROSS MACHINES



each machine holds DIFFERENT rows -> more capacity

✗ a query spanning shards gets much harder

##### REPLICATION · COPY THE SAME ROWS TO MANY MACHINES



same rows everywhere -> more read speed and a spare copy

✗ replicas can lag - a read may be slightly stale

Notice: sharding splits the data, replication copies it - most real systems do both.

Notice: sharding splits the data, replication copies it — most real systems do both.

### Common Confusion

The two get swapped constantly. Fix it with one sentence: **sharding gives each machine different rows; replication gives each machine the same rows.** Different versus same.

Then the follow-up: **sharding solves capacity, replication solves read load and redundancy.** A sharded system with no replicas still loses data when one machine dies — each shard is a single point of failure for its slice.

Second: **replication is not a backup.** A deleted row replicates instantly to every replica. Replication protects against a machine dying, not against a mistake — which is why Module 8.4's backups still exist.

### Interview Angle

1. "Difference between sharding and replication?" — Sharding splits rows across machines for capacity; replication copies rows for read speed and redundancy.
2. "What makes a good shard key?" — One that spreads traffic evenly and keeps related rows together. A bad key creates a hot machine and cross-shard queries.
3. "Why might a read return stale data after a write?" — The read hit a replica that hasn't caught up with the primary yet.
4. "Is replication a backup?" — No. Mistakes replicate too. Backups protect against bad data; replication protects against a dead machine.

**Recap:** Sharding splits rows across machines to add capacity, replication copies rows to add read speed and redundancy — and neither replaces a backup.

